UE5 Dataset Generator Plugin Guide

Markus Leinberger

July 2026

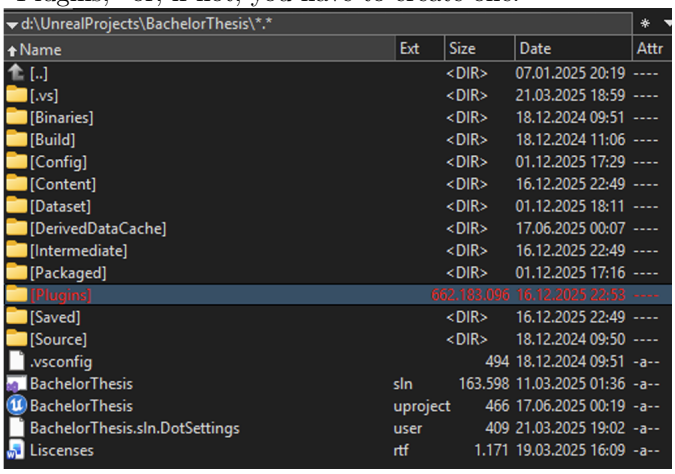
Contents

Based on the Guide from Gutbier (2025).

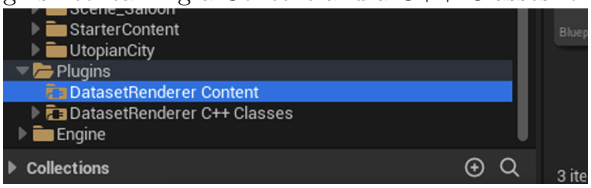
1 Plugin Installation

The Plugin installation can be done in a few quick steps:

1. Make a new empty UE5 Project (or use an existing one)
2. In the root directory of that project, you will either find a directory called "Plugins," or, if not, you have to create one.



3. In the "*Plugins*" directory, create a new one called "*DatasetRendererBachelorThesis*" and paste the content of the Plugin folder from the GitHub page.
4. Start UE5 and open the project.
5. Go to "*Edit− > Plugins*" inside the opened project, search for the plugin and check the checkbox.
6. (You may need to restart UE5)
7. Inside the content drawer, there should now be a new folder called "Plugins" containing a Content and a C++ Classes folder.



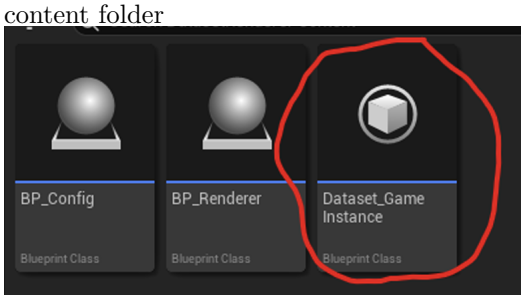
8. You can now start configuring UE5 and using the plugin

2 Engine Configuration

2.1 Set Custom Game Instance

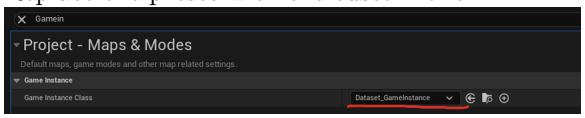
Before you get started creating your own dataset, you need to correctly configure the engine to ensure the best results.
First, we need to set the correct game instance. It holds all the global variables we need across levels to automatically create the dataset and load new levels after one is done rendering.

1. Make sure that there is a game instance Blueprint class in the plugins content folder



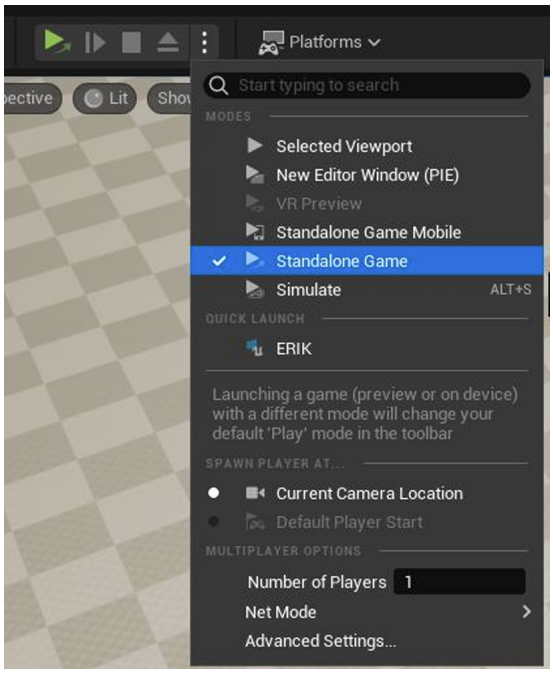
2. Go to "Edit− >Project Setting" and search for "Game Instance" in the search field.

3. Replace the preset with the custom one.



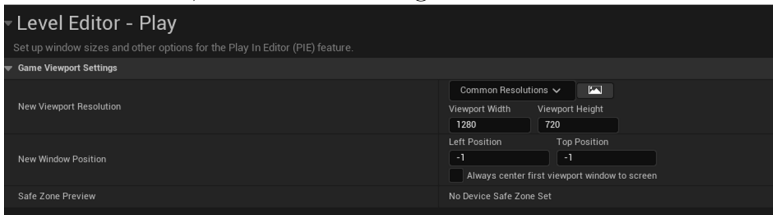
2.2 Set Player to Standalone Window

When using the in-engine Player to render, you need to set it to open in a standalone window.
Otherwise, you will run into issues where resources are not properly cleaned up after restarting the player and end up with duplicate metadata. For this, click the three dots next to the player and select "Standalone Game".



2.3 Set Window Resolution

Due to how the plugin captures images, the only way to control the resolution is by adjusting the size of the game window. For testing using the in-engine player, you can set this by going to "$Edit > EditorPreferences$" and searching for "$Viewportsettings$".
Here you can now set your preferred resolution for your images. The smaller the resolution is, the faster the images will render.

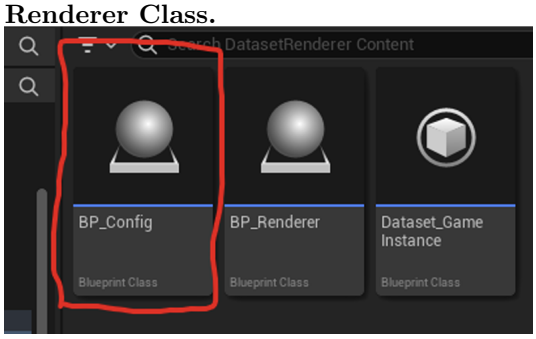


3 Render Components

3.1 Config

The config component is currently used to configure the render process, including setting the custom levels you want to render in and your 3D models. You only need it in one single starting Level. Just add it per drag and drop to the level viewer and then select it. In the scene element overview on the right side,

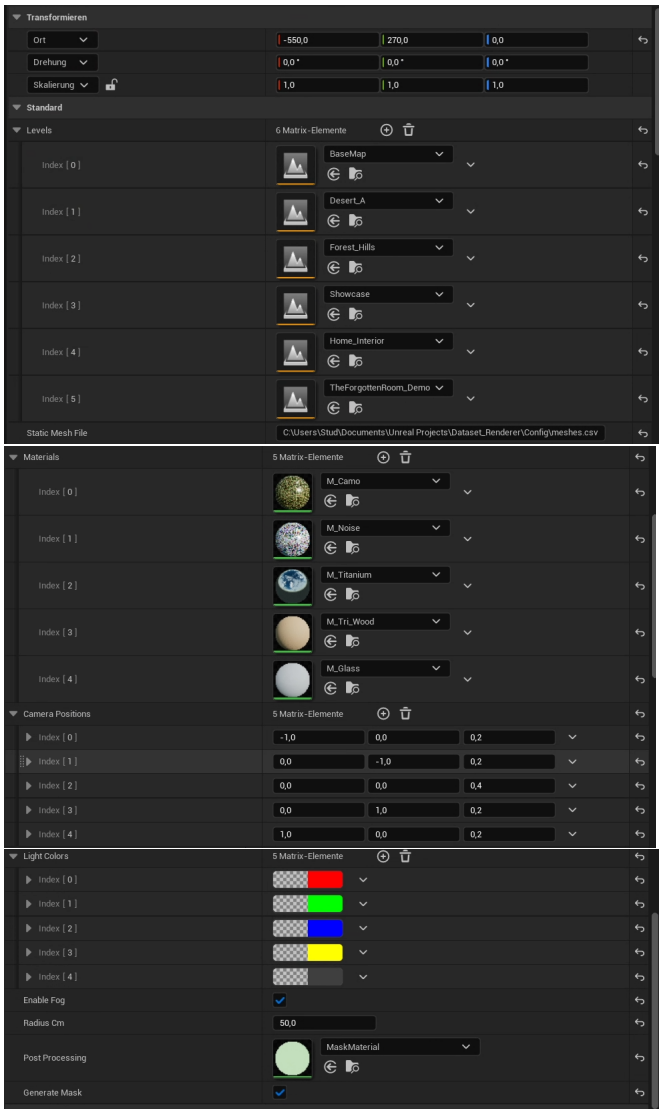
edit the settings.
To access and edit the Blueprint logic, just double-click the Blueprint class, and it will open inside an editor.
NOTE: The level containing the Config Class should not contain the Renderer Class.



How to adjust the factors:

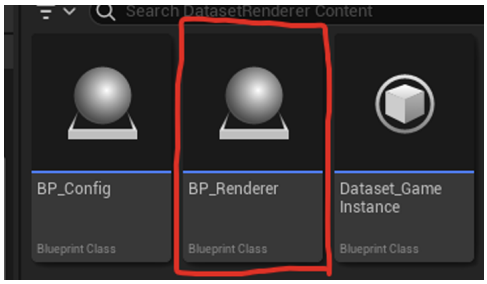
1. **Levels** – The UE Levels that serve as the background.
2. **Actors** – A map of actor that contain the static meshes to their respective class in the dataset.
3. **Materials** – Set any material. After an actor was rendered with its default materials whatever you set here will replace all default materials of the mesh and re-do the render process with the new materials.
4. **Camera positions** – Relative direction vectors (X, Y, Z). The system dynamically computes camera distance so the object is centered and well-framed.
5. **Light colors** – Different RGB colors for the scene lighting.
6. **Fog** – Fog.
7. **Radius** - All meshes are automatically scaled so their bounding sphere matches this value (default: 50cm), ensuring consistent proportions.
8. **ObjectFramePercentage** - Has to be set in the code itself. Default set to 60% to ensure objects occupy a specific ratio of the frame, providing padding for downstream tasks like patch extraction.
9. **Segmentation Masks** - To generate binary black-and-white masks, assign a material (like MaskMaterial) to the Post Processing field.

All possible combinations of these parameters will be rendered for each actor in each level.

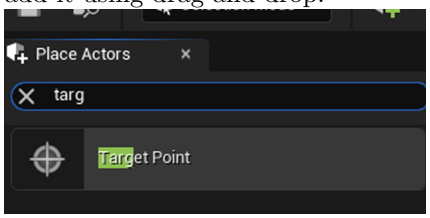


3.2 Render

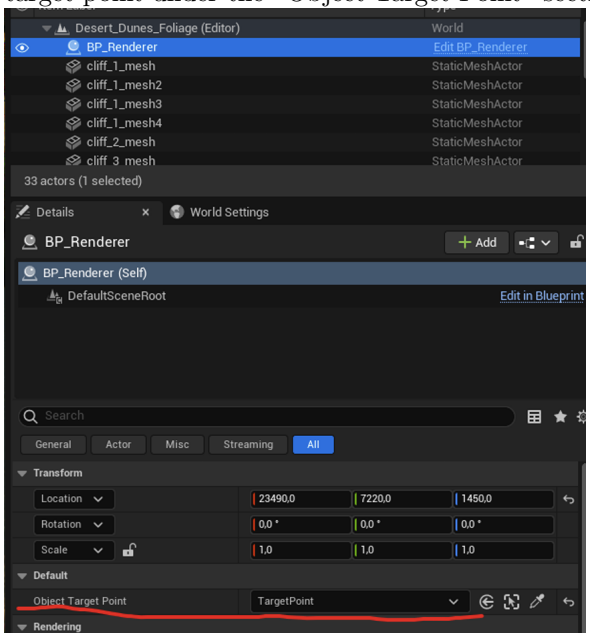
The render element needs to be added to each level you want to render (The levels you add to the config). This can again be achieved by loading the level into the editor and adding the Blueprint class via drag and drop.



To access and edit the Blueprint logic, just double-click the Blueprint class, and it will open inside an editor.
There is not much else to do except to add a target point to this level, which will serve as the spawn point for your Actors.
To add a target point to your scene, go to the Place Actors tab on the left and add it using drag and drop.

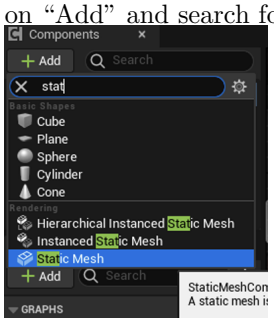


After you added it to your scene, select the renderer component and add your target point under the “Object Target Point” section.

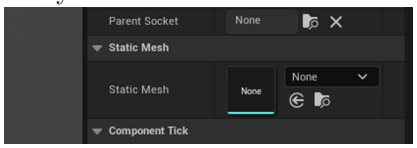


3.3 Actors

With the current version, all your static meshes need to be added to a UE5 actor before being added to the config class. For this, go to any folder inside the Content Drawer and create a new Actor using "$rightclick- > BlueprintClass- > Actor$".
Double-click your new actor, and on the left side in the "Components" tab, click on "Add" and search for "Static Mesh".



Afterward, select that new element and, on the right side, in the Details tab, add your static mesh to the actor.

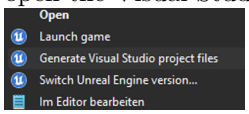


Then you can adjust the size and rotation inside the preview. When you are done don't forget to save and compile the new Actor before closing out.

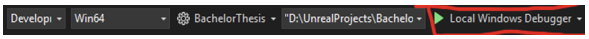
4 C++

If you want to adjust the C++ part of the plugin, I recommend installing the visual studio extension for unreal engine: https://dev.epicgames.com/documentation/en-us/unreal engine/setting-up-visual-studio-development-environment-for-cplusplus-projects-in-unreal engine

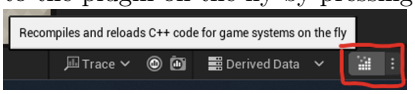
After you set it up following the official guide, you should make sure that UE5 is not running. Then go to your project root, right-click on the *projectname*.uproject, and select "Generate Visual Studio project files". Then open the Visual Studio project.



Note that re-generating the project files is always the first troubleshooting step when you get linker errors or other esoteric issues when compiling.
Once everything loads, you can start the engine by starting the local Windows debugger.

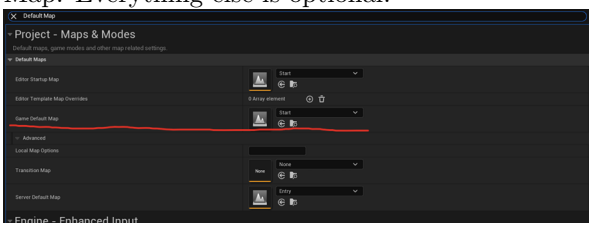


This will start UE5 with your project loaded and allow you to do code changes to the plugin on the fly by pressing one button on the bottom right of UE5.



5 Rendering

Once everything is set up and you have your first Levels and Actors, you can try to do a test render. However, first you must set your Start Level with your config element as the Game Default map. For this go to "$Edit- > ProjectSettings$" and search for "Default Map" and set your Starting level as the Game Default Map. Everything else is optional.



The plugin automatically waits for all level and mesh textures to be fully resident before capturing.
The capture process is structured into explicit phases, separating variations in camera, lighting, material, and fog into distinct rendering stages.

Now you can press the green play button in the editor and run the whole thing.

1. **Output Structure** - Images and masks are organized into subfolders by the factors: $< ProjectRoot > / < Dataset > / < Factor - Name > / < Factor - Variation > / < Image >$.
2. **Metadata** - A Metadata.csv file is generated in the Dataset folder including a Mask column.
3. **Important Note** - The system does not overwrite existing files. It appends to the CSV and adds numbers to duplicate image names. Manually delete or move old "$Dataset$" folders before starting a new run.
4. **State Restoration** - The adapted pipeline automatically preserves and restores the scene's original lighting and material state between runs to prevent unintended accumulation of changes.